

QA5 物件導向程式設計課堂筆記

涵蓋章節： CH5 (C++ vs Java 比較)、CH6 (軟體工程與程式碼品質)、Ch7 (物件導向與資料庫設計)

CH5: C++ 與 Java 物件導向語法比較

✦ 背景前言：為什麼這門課實作用 Java？

- **C++ 的歷史包袱**: C++ 是較早引入物件導向特性的語言，當初為了將所有 OOP 概念納入，導致語言變得「又肥又大」，早期的編譯器執行速度慢，且學習門檻極高（如複雜的指標、多重繼承造成的歧義等）。
- **Java 的改良**: Java 是較晚設計的語言，它吸收了 C++ 的慘痛經驗，對物件導向的語法進行了大量精簡與美化，拿掉了容易出錯的機制。這也是為什麼本課程在實作標準上選擇使用 Java 的原因。

1. 建構子 (Constructor) 與父類別參數傳遞

當子類別需要將參數傳遞給父類別（基底類別）時，兩者的底層邏輯與寫法有顯著差異：

- **C++ 的作法**: 成員初始化列表 (Member Initialization List)
  - C++ 利用建構子宣告的冒號 `:` 到大括號 `{}` 之間來進行變數初始化或呼叫父類別的建構子。
  - 原理: 大括號內是建構子真正執行的程式碼，而在進入大括號前，編譯器會先透過這個「初始化列表」將參數傳遞給父類別，確保父類別的建構子優先被呼叫。
  - 簡報範例: `Rat(int w, string f) : Pet(w,f) {}`。
- **Java 的作法**: `super()` 關鍵字
  - Java 沒有初始化列表機制，而是直接在建構子中使用 `super()` 來呼叫父類別（上一層 parent class）的建構子。
  - 嚴格限制: `super()` 必須是建構子內的第一行程式碼，不能出現在第二或第三行。如果寫在其他地方會違反編譯器建立物件與呼叫建構子的先後順序，導致編譯錯誤。
  - 另一個限制: 在 `super()` 內，不能使用 `this.xxx` 存取任何自身的變數或方法來計算傳入的參數。
  - 簡報範例: `Rat(int w) { super(w); }`。

2. 陣列宣告與記憶體配置 (Array & Object Allocation)

這點是 C++ 與 Java 差異最大的地方，初學者極容易混淆：

- **C++**: 直接配置記憶體並建立物件
  - 宣告語法如 `Pet allpets[100]`；時，編譯器會直接配置 100 個物件的記憶體空間，並且連續呼叫 100 次建構子。
- **Java**: 僅建立參考 (Reference)
  - 宣告同樣的 `Pet allpets[100]`；時，Java 只會建立 100 個參考（類似 C++ 的指標），完全沒有配置任何實際的物件記憶體空間。
  - 若要真正建立物件，必須透過迴圈（如 `for` 迴圈），逐一使用 `new Pet()` 將實體指派給這些參考。
- **Java 的基本型別與類別型別差異**:
  - 原始型別 (Primitive type): 如 `int`, `boolean`, `float`, `double` 等。當宣告 `int x`；時，Java 會真的配置記憶體（例如 4 bytes）並建立變數。
  - 類別型別 (Class type): 如自訂的 `Pet`。只要不是原始型別，宣告時都只會建立 reference，不會產生真正的變數實體。老師特別提醒，初學 Java 必須釐清這點，否則會一團混亂。

3. 方法的參數傳遞 (Parameter Passing to Methods)

參數傳遞的方式深刻影響了程式的執行效能與安全性：

- **C++**: 彈性但暗藏效能陷阱
  - 所有基本型別或類別型別，都可以使用傳值 (pass by value)、傳址 (pass by address) 或傳參考 (pass by reference)。
  - 效能問題: 若對包含複雜資料結構的超大物件使用「傳值」，C++ 會觸發「拷貝建構子 (copy constructor)」，將龐大的記憶體完整複製一遍，這是導致 C++ 程式常被詬病「效能差」的主要元兇（通常是因為程式設計師寫法不良所致）。
- **Java**: 為了安全與效能做出嚴格限制
  - 原始型別 (Primitive Types): 只能傳值 (Only pass by value)。
  - 類別型別 (Class Types / Objects): 只能傳參考 (Only pass by reference)。Java 這樣設計是為了避免無意中複製龐大物件所帶來的效能災難。
  - 🧐 老師的思考題: 在 Java「基本型別只能傳值」的嚴格限制下，如果想要寫一個 Function 來交換兩個變數 (a 與 b) 的數值，該怎麼寫？（請同學自行思考）。

4. 變數宣告與物件配置區域 (Object Allocation Scope)

| 類型   | C++                           | Java                                       |
|------|-------------------------------|--------------------------------------------|
| 全域變數 | 存放於 data segment, 適用基本型別與類別型別 | 不允許在 Class 外宣告; 需用 <code>static</code> 關鍵字 |
| 區域變數 | 存放於 stack                     | 只有原始型別 (primitive types) 可作為區域變數           |
| 堆積變數 | 適用所有動態配置的型別                   | 所有物件一律透過 <code>new</code> 建立在 Heap 中       |

- 🧑‍🏫 老師的警告: 很多剛轉寫 Java 的人因為不習慣，會為了避開編譯錯誤而濫用 `static`；「`static` 用得越多，代表越不懂物件導向」，這只是一種想回到 C/C++ 舊習慣的寫法。

5. 抽象類別 (Abstract Class)

抽象類別用來定義介面，無法用來直接建立物件實體：

|       | C++                                    | Java                                |
|-------|----------------------------------------|-------------------------------------|
| 語法    | <code>virtual void speak() = 0;</code> | <code>abstract void speak();</code> |
| 名稱    | Abstract class （早期稱 virtual class）     | Abstract class                      |
| 嘗試實例化 | 編譯器報錯                                  | 編譯器報錯（「會過就有鬼」）                      |

- **Java 規則**: 只要類別中有任何一個 method 被標示為 `abstract`，該類別本身就必須是抽象類別。

6. 繼承中 `super` 的進階應用 (Overridden Methods)

當子類別覆寫 (override) 了父類別的方法，但又需要呼叫父類別原本的實作時：

- **Java**：使用 `super.methodName()`
  - 除了在建構子使用 `super()`，`super` 的另一個意義是作為「父類別的參考 (parent class's reference)」。
  - 簡報範例：若 `myMethod` 被覆寫，可以使用 `super.myMethod(font)`；來呼叫父類別原本的版本。
- **C++** 的對應語法
  - 在 C++ 中若要達到同樣目的，必須明確指定基底類別名稱，例如寫成 `Base::doSomething()` 來呼叫被覆寫掉的原始函數。

CH6：軟體工程與程式碼品質 (Source Code Quality)

一、軟體的本質與挑戰

與其他傳統工程（如機械、建築）不同，軟體開發具有以下獨特的本質：

- **抽象 (Abstract) 與難以描述 (Hard to describe)**：軟體目前沒有像機械工程圖那樣標準、讓大家「按圖施工」的圖紙。
- **不可見性 (Invisible)**：你可以輕易透過視覺判斷一面牆漆得好不好、機械零件的密合度高不高，但你無法用肉眼直接看出幾千行程式碼的架構與品質好壞。程式碼不是幾何圖形，必須花費大量時間去理解 (Program Understanding) 才能知道其耦合與依賴關係。
- **動態行為 (Dynamic behaviors)**：程式碼本身是靜態的文字，但執行起來具有動態行為，增加了理解與評估的難度。
- **藝術與科學的成分**：
  - **藝術**沒有客觀的評估標準，只能意會不能言傳，通常透過「鑑賞」來學習。
  - **科學**則基於客觀標準、可重複的實驗，並能用數學等形式化語言描述。
  - 寫軟體帶有藝術的成分，傳統資工教育缺乏「程式碼鑑賞 (Code Review)」的訓練，導致學生不知道什麼是好程式碼、什麼是壞程式碼。

二、高手 (Peak Performer) 與庸手 (Mediocre Programmer) 的差異

- **十倍的差距**：軟體工程研究顯示，頂尖高手與平庸程式設計師的產出與品質表現，往往存在一個數量級 (10倍) 的極大差異。
- **年資不等於能力**：成為高手與工作年資、經驗沒有絕對關係。
- **解決問題的態度**：庸手花了一個月解完 Bug，下次遇到類似問題會認為「花一個月是正常的」；而高手會去尋找工具與方法，探究為何要花這麼久，下次遇到時可能半天就能搞定。
- **【真實案例】忽視流程的軍中 SOP**：老師舉例在飛彈指揮部當兵的同學，發現原文書操作手冊與部隊流傳的 SOP 不符。但士官長卻說「沒關係，我們過去十幾年都是這樣做的」。這說明許多工程師習慣沿用過去錯誤的方法做事而不自知，年資深不代表做法正確。

三、壞程式碼 (Bad Code) 的特徵與成因

壞程式碼的特徵

1. **難以理解 (Difficult to understand)**：變數與邏輯混亂，必須不斷重新探索 (Rediscovery) 才能看懂。
2. **牽一髮動全身 (Difficult to change)**：修改一個地方，會產生連鎖效應 (Ripple effect)，導致其他無關的地方也跟著壞掉或必須修改。
3. **寫死解決方案 (Hardwire the solutions)**：遇到問題不去想好的演算法或資料結構，而是用一堆 `if/else` 硬幹。
4. **高度耦合 (Tightly coupled)**：過度使用全域變數 (Global variables)，導致各模組緊密糾纏。
5. **霍爾的移動城堡 (疊床架屋)**：缺乏設計的程式碼就像東拼西湊的城堡，隨便拆除一個舊功能（例如砲塔），整個系統架構就可能崩壞。

為什麼會寫出壞程式碼？

1. **資工教育的落差 (CS Curriculum)**：大學課程過度專注於「解決問題 (Problem solving)」如演算法、新技術，卻不重視程式碼品質。這就像「只教你怎麼把齒輪組裝起來解決問題，卻不教你怎麼打磨出高品質的齒輪，也不教你怎麼評估齒輪好壞」。
2. **時程壓力與預算有限 (Tight schedule & limited budget)**：現實中經常因為死線逼近而妥協。
3. **Demo 導向編程 (Demo-oriented programming / Demo or Die)**：為了應付展示，先拼湊出會動的程式，事後卻沒時間重新設計 (Refactor)，導致系統一路「疊床架屋」發展下去。
4. **缺乏軟體工程流程與團隊文化**：
  - **【真實案例】台積電的痛點**：台積電早期將軟體專案當成硬體專案管理，切分給各個工程師獨立開發 (孤膽俠客 Lone Ranger)。大家各寫各的，沒有 Code Review，當工程師離職或轉調時，留下的程式碼完全無法維護與擴充，導致後續修改代價極高。這證明了建立「寫出讓別人看得懂的程式碼」的團隊文化至關重要。

四、程式碼鑑賞：Code Review 與品質衡量

- **什麼是 Code Review?** 在整合前 (Code Inspection) 或測試後，對提交的程式碼進行檢查與審查的過程。
- **品質的客觀衡量標準**：業界有一個幽默卻真實的指標：**WTFs/Minute** (每分鐘罵髒話的次數)。好的程式碼 Code Review 時很平靜，壞的程式碼會讓人不斷發出 "WTF" 驚呼。

五、軟體架構的反模式 (Anti-Patterns)：麵食理論

老師使用義大利麵食來比喻軟體結構的好壞：

| 麵食類型                  | 說明                                                                         | 評價   |
|-----------------------|----------------------------------------------------------------------------|------|
| 義大利麵 (Spaghetti Code) | 缺乏文件，結構糾結不清，難以擴充或修改。常見症狀：無人維護、依賴搜尋工具找變數、過多全域變數、複製貼上、過深的 if/for 嵌套          | 最糟   |
| 千層麵 (Lasagna Code)    | 有分層架構且容易理解，但過於龐大單體化 (Monolithic)。修改其中一層非常困難，層與層之間久了還是會牽扯不清                 | 次差   |
| 義大利餃 (Ravioli Code)   | 每個元件就像一顆顆獨立的餃子，「體積小且鬆散耦合 (Small and loosely coupled)」。可輕鬆替換或修改其中一顆，不影響整盤系統 | 理想境界 |

六、軟體設計的核心目標：解耦合 (Decoupling)

軟體開發的夢想是達到：乾淨簡單的介面 (Clean and simple interface)、可替換性 (Replaceability)、模組化 (Modularity) 以及可組合性 (Composability)。

- **耦合度 (Coupling)** 是關鍵：
  - 高耦合是壞的，我們追求的是低耦合。
  - 物件導向 (OO) 或軟體工程的各種方法，本質上都是一種「解耦合」的技術，確保你「修改、替換、擴充」一個東西時，不需要牽連修改其他一堆東西。

七、程式碼異味 (Code Smell) 與重構原則

1. Curly's Law（做一件事法則）

- **Do One Thing**：任何一段程式碼都應該只有一個明確的目標。這點非常困難，因為開發者常有把功能塞在一起的傾向。
- 此原則呼應了 **DRY (Don't Repeat Yourself)**、**Once and Only Once**（每個行為只該出現一次）以及 **Single Point of Truth**（單一真相來源），避免重複宣告導致後續維護的災難。
- **抽象層級的一致性**：類別設計時，不應混合高低不同層級的抽象（例如將 `List` 容器底層操作與 `Employee` 員工高層操作寫在同一個類別中），且應避免維護過程中功能不斷膨脹、失去焦點。

2. 複製貼上程式設計 (Copy-Paste Programming)

- David Parnas 指出：只要你寫程式用到複製貼上，你大概就犯了設計錯誤。應該將重複的程式碼抽出成為獨立的 Function，以確保未來修改只需在單一地點進行。
- **從網路抄 Code 的利弊**：雖然容易找到並有 Pagerank 把關，但壞處是盲目複製會無法理解其運作、無法接收原作者的更新，且網路範例常是忽略錯誤處理與資安考量的「Demo Code」。

3. 常見的程式碼異味 (Code Smells) 清單

若程式出現以下狀況，代表設計可能出了問題，需要重構 (Refactor)：

- 方法/類別過於龐大： Long Method（過長函式）、Large Class（過大類別）、Long Parameter List（參數列過長）。
- 命名不佳： Uncommunicative Name（無法溝通的命名）、Inconsistent Names（命名規則不一致，如 Queue、Q 混用）、Type Embedded in Name（型別寫入變數名中）。
- 邏輯問題： Duplicated code（重複程式碼）、Conditional Complexity（條件邏輯太複雜，應考慮策略或狀態模式）、Dead Code（未使用的死板程式碼）。
- 類別間的異味：
  - *Primitive Obsession*（過度依賴基本型態而不建類別）
  - *Data Class / Data Clumps*（只有資料沒有行為的類別 / 總是成群結隊出現的資料）
  - *Feature Envy / Inappropriate Intimacy*（方法過度羨慕/依賴別的類別的屬性、類別間過度親密）
  - *Middle Man*（只負責轉交呼叫的冗餘中介類別）
  - *Shotgun Surgery*（散彈槍外科手術：修改一個小功能，必須同時修改多個類別的片段）
- 非區域性計畫 (Non-localized plans)： 新增一個功能卻需要更改系統的許多部分（例如平行的類別繼承體系，新增 A 就要跟著新增 B），解決方法是將一起變動的方法放在一起。

Ch7：物件導向與資料庫設計

壹、為何在物件導向課程中談論資料庫？

老師開宗明義提到，雖然這是一門物件導向（OO）的課程，但物件導向背後的理論其實與資料庫（Database）有著非常緊密、深刻的關聯。在未來設計系統時，動輒需要使用資料庫，若對資料庫沒有足夠的理解與良好的設計，未來會遇到很多難以維護的麻煩。老師認為資工系的學生即使未來主要做演算法或系統開發，仍應該具備資料庫的基本設計能力，因為系統的背後多半是在處理「人」的資料與商業邏輯。

貳、資料庫歷史與關聯式模型 (Relational Database)

- 早期資料結構： 在大約五、六十年前，為了儲存人類的資料，許多商用產品使用了複雜的資料結構，如 Tree（樹狀圖加上指標）、Graph（圖形結構）與 Linked list。早期的資料模型包含了 Hierarchical model（階層式模型）與 Network model（網狀模型）。
- 關聯式模型的誕生： 1970 年代，大師 Edgar F. Codd 提出了一篇顛覆性的論文，主張應用程式應透過內容（關聯）來尋找資料，而不是透過複雜的結構連結。時至今日，市面上大多數的資料庫產品都採用這種 Relational model（關聯式模型）。
- 核心名詞對應關係：

| 術語一             | 術語二           | 術語三   | 說明                        |
|-----------------|---------------|-------|---------------------------|
| Table（表）        | Relation（關聯）  | —     | —                         |
| Row（列）          | Record（紀錄）    | Tuple | 代表一筆完整的資料                 |
| Column（欄位）      | Attribute（屬性） | Field | —                         |
| Primary key（主鍵） | —             | —     | 能唯一區分兩筆資料的欄位（如身分證字號、員工編號） |
| Foreign key（外鍵） | —             | —     | 來自其他表格的主鍵，用於關聯兩個表格        |

參、業餘資料庫設計師的常見錯誤

當接到一個需要產生「銷售月報表」（包含顧客名稱、地址、產品型號、單價、數量、總價等）的需求時，未經訓練的「業餘 (Amateur)」設計師常會犯一個大錯：直接建立一個完全模仿報表欄位的單一關聯式表格。

- 根本問題所在： 這樣的一個報表其實包含了三種完全不同層級的資料：描述顧客的資料 (Customer)、描述產品的資料 (Product) 以及描述銷售動作的資料 (Sale)。
- 維護的噩夢 (Maintenance horror)： 將全部資料塞進同一個表，會產生大量的「冗餘資料 (redundant data)」。如果顧客改了地址，或者某產品的單價變動了，系統必須寫程式（例如跑一個 for 迴圈）把所有的歷史紀錄全部找出來並修改，導致極度沒有效率，且容易發生資料不一致的問題。
- 最可怕的現象 (Unanticipated Queries)： 當這種未經「正規化」的資料庫運行幾年後，如果遇到了設計師當初沒有預料到的查詢需求，透過簡單的 SQL 語法將無法達成，必須仰賴額外寫程式才能撈出資料，這就是資料庫領域最害怕的現象。

肆、正規化 (Normalization) 的核心目標與階層

正規化的定義是將關聯式資料庫的綱要 (schema) 結構化，藉此消除大多數的歧義，確保每個資料表只包含真正用來描述該實體所需屬性。

- 兩大目標：
  1. 消除冗餘資料 (Eliminate redundant data)： 避免同樣的資料重複儲存。
  2. 確保資料相依性合理 (Ensure data dependencies make sense)： 只把有相關的資料儲存在對應的表格中，節省空間並保持邏輯正確。
- 正規化階層： 由最寬鬆的第一正規化 (1NF) 一路檢驗到第五正規化 (5NF)。實務上，大部分設計師通常只會做到第三正規化 (3NF) 或 Boyce-Codd 正規化 (BCNF)，偶爾做到 4NF，幾乎不會用到 5NF。

伍、正規化的實作步驟與範例

一、第一正規化 (First Normal Form, 1NF)

- 定義： 消除同一表格中的重複欄位；為每一組相關資料建立獨立的表格，並給予一個能唯一辨識每一列的主鍵。
- 範例探討 (經理與下屬的關係)： 假設商業規則是「每個經理可擁有多名下屬，但每位下屬只能有一位經理」。
  - 直覺寫法一： 開設 Subordinate1、Subordinate2... 等多個欄位。這會浪費大量未使用的儲存空間，且萬一下屬數量超過預設欄位數（例如 255 個），整個資料庫結構就必須停機重整。
  - 直覺寫法二： 建立一個字串欄位，將下屬名字用逗號隔開（例如 "Jim, Mary, Beth"）全部塞進去。這種寫法不符合「不可分割 (atomic)」原則，未來新增或刪除下屬時，必須讀出整個字串進行字串處理，效能極差。
  - 符合 1NF 的設計： 將關係拆解成一筆一筆（如 Bob-Jim, Bob-Mary），確保只有縱向發展而不會橫向擴展。但老師提醒，不該使用「名字」作為主鍵（因為會改名或同名同姓），應該使用永遠不會更動的員工編號 (ID) 來取代名字，並另外建立一個存放人員 ID 與姓名對應的 Person 表格。

二、第二正規化 (Second Normal Form, 2NF)

- 定義： 必須先滿足 1NF。接著，表格中的每個非鍵屬性 (nonkey attribute) 必須「功能相依 (functionally dependent)」於主鍵。要移除適用於多列資料的子集，將它們獨立成新的表格。
- 範例探討 (訂單與聯絡人)： 一張訂單包含：Order ( # (主鍵)、Customer (顧客)、Contact Person (負責該顧客的業務)、Total (總額)。
  - 問題： Contact Person 其實是依賴 Customer，而不是依賴這張訂單 (Order ( # ) )。如果該顧客的負責業務換人了，所有歷史紀錄中的聯絡人都要修改。
  - 關鍵觀念： 判斷「功能相依性」不能單靠死板的數學定義，而是必須仰賴人類的「商業經驗與文化邏輯」。例如，如果該業務是負責「這間公司」，那就該拆表；但如果在另一種文化中，業務是綁定「單筆訂單」，那目前的表格就是對的。
  - 解法： 拆成兩個表。第一個表存放 Order ( #、Customer ID 與 Total；第二個表存放 Customer ID 與 Contact Person。未來查詢時，資料庫會透過 Foreign Key 執行 JOIN 操作來組合資料。換業務員時，只需在第二個表修改一筆資料即可。

三、第三正規化 (Third Normal Form, 3NF)

- 定義： 必須先滿足 2NF。此外，所有非主鍵屬性必須不能遞移相依 (non-transitively dependent) 於候選鍵。白話來說，就是非鍵屬性之間不能有依賴關係。
- 範例探討 (公司地址與郵遞區號)： 表格包含 Company、City、State、ZIP (郵遞區號)。
  - 問題： 城市 (City) 與州別 (State) 其實是由郵遞區號 (ZIP) 決定的。只要知道郵遞區號，就等同於知道了城市與州。
  - 解法： 拆成兩個表，一個儲存 (Company, ZIP)，另一個儲存 (ZIP, City, State) 的對應關係。

- **權衡 (Trade-offs)**：往上做到越高等級的正規化，表格切得越多，能應付的未來查詢 (Query) 彈性就越高。但如果過度切分，過多的 JOIN 操作會吃掉系統效能。設計師需在靈活性與效能間取捨 (例如：如果地名一千年都不會變，其實不用硬拆，資料重複也無妨)。

## 陸、物件導向設計與資料庫正規化的呼應

老師在課程後段將資料庫設計對應回物件導向程式設計 (OOP)。

- 如果在宣告物件類別 (Class) 時，將客戶的業務聯絡人 (Contact Person) 直接當作屬性寫死在「Order」物件中，這種行為就等同於未正規化。
- 當業務聯絡人更換時，你的主程式必須寫一個迴圈，去掃描記憶體中上百萬個 Order 物件來做字串替換，這會是一場災難。
- **優良的物件導向分析**：應該像資料庫設計一樣，將這層關係抽離出來，建立獨立的關係類別 (例如 `customerContactRelation`)，並利用指標/參考 (Pointer/Reference) 來關聯資料。這樣一來，改動資料時「只要改一個地方，不用改很多地方」，這正是物件導向與資料庫正規化共同的核心精神。

## 柒、課堂小練習 (Exercise)

老師在簡報最後留下了一個系統分析練習題，要求分析一個包含零件供應商與零件資料的系統：

- **包含欄位**：S# (供應商編號)、SNAME (供應商姓名)、CITY1 (供應商城市)、P# (零件編號)、PNAME (零件名稱)、COLOR (零件色彩)、WEIGHT (零件重量)、CITY2 (零件儲存城市)、QTY (零件存量)。
- **條件**：一個零件可以由多個供應商供應。
- **問題**：請求出這個資料庫系統應該被切分為「幾個表格」，以及每個表格的屬性為何？請同學自行思考。